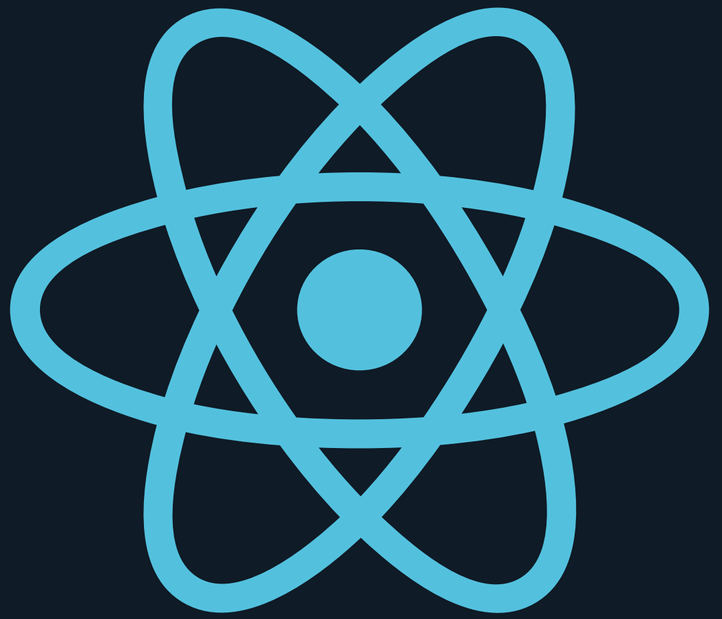
SEMINÁRIO

React: uma biblioteca para interfaces



Docente: Humberto Lidio Antonelli
Discentes: Luis Botelho, Pedro Rocha, Vitor Hugo Bonfim

Sumário do seminário

01 **Introdução ao React**

História, filosofia, adoção e ficha comparativa

02 **Fundamentos**

JSX, componentes, props, estado, efeitos, rotas e Context

03 **Live coding**

RF1 e RF2: listagem de eventos, busca e filtros

04 **Oficina em duplas**

RF3, RF4 e RF5: cadastro, detalhes e interesses

05 **Fechamento**

Quiz, perguntas e síntese dos conceitos

React é uma biblioteca JavaScript para criar interfaces de usuário.

- Organiza a tela em componentes reutilizáveis.
- Atualiza o que aparece quando os dados mudam.
- Pode ser adotado em partes de um site ou em aplicações completas.

Ideia central: a interface acompanha o estado da aplicação.



Contexto histórico

Como o React surgiu?

Facebook

Desenvolveu React para lidar com interfaces dinâmicas.

2013

React foi disponibilizado como código aberto.

Expansão

A comunidade passou a criar bibliotecas, ferramentas e frameworks ao redor dele.

Hoje

React é utilizado para interfaces web e, com tecnologias relacionadas, também para aplicações móveis.

O desafio era organizar interfaces que mudam frequentemente sem manipular manualmente cada elemento da página.

Filosofia do React

Construir a interface com componentes

Uma página pode ser dividida em partes menores:

- Cabeçalho
- Lista de eventos
- Card de evento
- Formulário

Interface declarativa

Em vez de escrever cada alteração visual manualmente, descrevemos como a tela deve aparecer para um determinado estado.



Biblioteca ou framework?

React cuida da camada de interface. Para aplicações completas, a equipe pode combinar outras soluções para rotas, organização e acesso a dados.

- Rotas
- Organização
- Acesso a dados

Quem usa React e por quê?

Exemplos conhecidos



Facebook e Instagram

Aplicações relacionadas à origem e à adoção inicial do React.



Khan Academy, Airbnb e The New York Times

Exemplos de adoção registrados pela equipe React desde os primeiros anos.



Projetos próprios

Lojas virtuais, painéis administrativos, plataformas educacionais e sistemas de eventos.

Por que escolher?



Reutilização de componentes.



Organização de interfaces complexas.



Atualização da tela a partir do estado.



Grande ecossistema de ferramentas.

Ficha comparativa do React

Item	React
Natureza	Biblioteca JavaScript para interfaces. Roteamento e outras funções podem depender de ferramentas complementares.
Templates	Usa JSX, uma sintaxe semelhante ao HTML dentro do JavaScript, com expressões e condicionais.
Reatividade	Atualizações de estado solicitam novas renderizações; React calcula e aplica as mudanças necessárias na interface.
Estado local e global	useState e useReducer para estado local; Context API para compartilhamento. Bibliotecas externas também podem ser usadas.

Ficha comparativa do React

Item	React
Roteamento e formulários	React Router para navegação; formulários controlados com estado ou bibliotecas especializadas.
Criação de projeto	Vite pode criar a base de uma SPA, com arquivos como index.html, src/main.jsx e src/App.jsx.
Curva de aprendizado	Componentes e JSX são acessíveis com conhecimento de JavaScript; estado, efeitos e arquitetura exigem mais prática.
Quando escolher	Interfaces interativas, projetos com componentes reutilizáveis e equipes que valorizam flexibilidade de arquitetura.

React: a tela acompanha os dados

UI = f(estado)

Se o valor de um contador muda, o número que aparece na tela também muda.



Esta é a ideia que conecta os conceitos do seminário.

Nosso exemplo: Mural de Eventos

Vamos usar a mesma aplicação ao longo da apresentação: uma página que mostra eventos do curso.

Depois, essa aplicação também terá:

- **Filtros**
- **Cadastro**
- **Detalhes**
- **Botões de interesse**



JSX: como descrevemos a tela

JSX permite escrever uma estrutura parecida com HTML dentro do JavaScript.

CÓDIGO

```
const nome = 'Pedro';
// JSX no componente
return <h2>Olá, {nome}!</h2>;

// JavaScript gerado (simplificado)
jsx('h2', {
 children: `Olá, ${nome}!`
});
```

RESULTADO VISUAL

Olá, Pedro!

O navegador não executa JSX diretamente: o build o transforma em chamadas JavaScript. A função jsx é inserida pela ferramenta. As chaves recebem expressões.

JSX tem algumas regras próprias

JSX se parece com HTML, mas possui diferenças de nomes e sintaxe.

EM HTML

```
<div class="evento">
  <label for="titulo">Título</label>
</div>
```

EM JSX

```
<div className="evento">
  <label htmlFor="titulo">Título</label>
</div>
```

`class` → `className`

`for` → `htmlFor`

Tags fechadas + um elemento raiz



Em JSX, também fechamos as tags e retornamos um elemento raiz ou um fragmento.

Condições: mostrar o conteúdo certo

A interface pode apresentar conteúdos diferentes dependendo de uma condição.

CÓDIGO

```
{eventos.length === 0 && (
  <p>Nenhum evento encontrado</p>
)}

{marcado
  ? <p>Remover interesse</p>
  : <p>Tenho interesse</p>}
```

SE A LISTA ESTIVER VAZIA

Nenhum evento encontrado

&& mostra um elemento quando a condição é verdadeira. O ternário escolhe entre duas alternativas.

Listas: um card para cada evento

Com map(), cada objeto de uma lista pode gerar um elemento na página.

CÓDIGO

```
eventos.map(evento => (
  <CardEvento
    key={evento.id}
    evento={evento}
  />
))
```

RESULTADO VISUAL

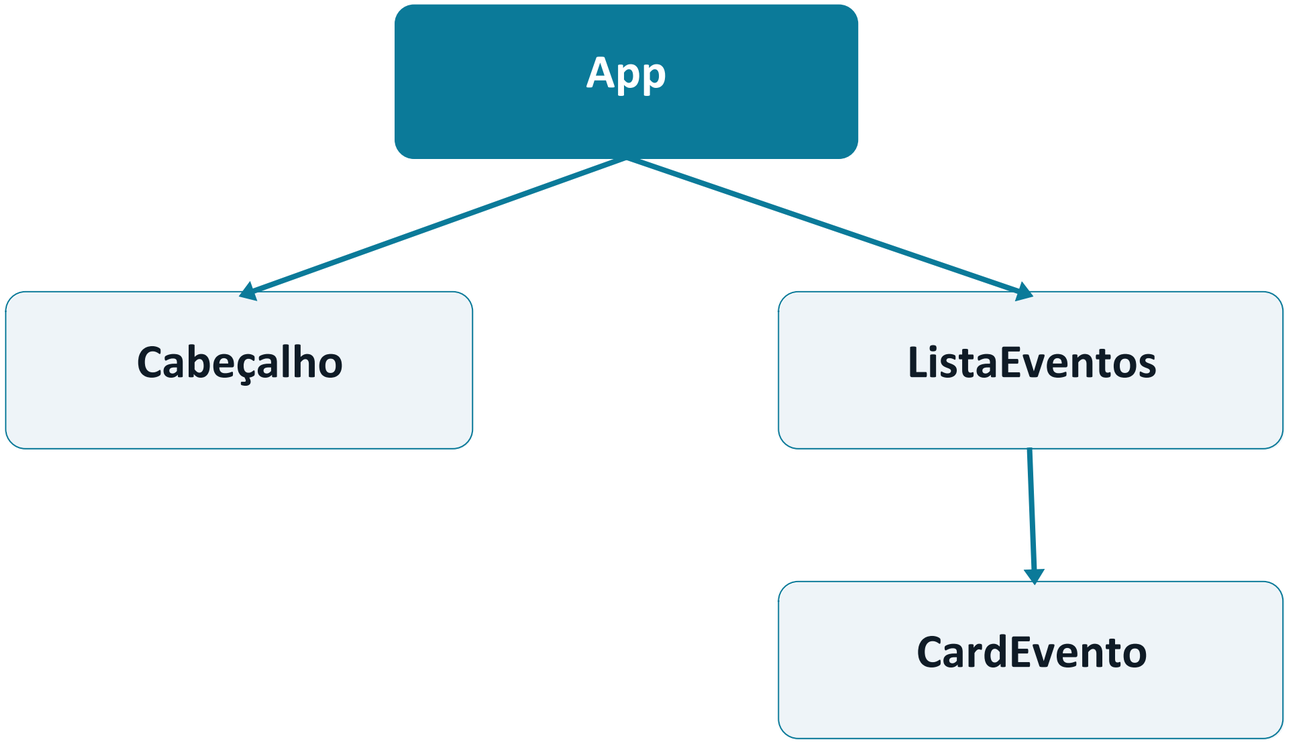
Card: Palestra de IA

Card: Workshop React

Card: Hackathon

key é uma identificação estável que ajuda o React a acompanhar cada item da lista.

Componentes: peças da interface



Um componente é uma função JavaScript que descreve uma parte da página.

Outras partes da aplicação:

`FormEvento`

`DetalheEvento`

Dividir a interface em componentes ajuda a organizar e reutilizar o código.

Props: passando informações

O mesmo componente pode mostrar conteúdos diferentes, recebidos por **props**.

CÓDIGO

```
function CardEvento({ titulo }) {
 return <h2>{titulo}</h2>;
}
<CardEvento titulo="Palestra de IA" />
<CardEvento titulo="Workshop de React" />
function Painel({ children }) {
 return <section>{children}</section>;
}
<Painel>
 <CardEvento titulo="Workshop" />
</Painel>
```

RESULTADO VISUAL

Card 1: **Palestra de IA**

Card 2: **Workshop de React**



Props são somente leitura. Painel envolve outro componente e recebe seu conteúdo por children: isso é composição.

useState: dados que podem mudar

O estado (state) é a memória do componente. Quando mudamos esse valor, o React atualiza a interface.

CÓDIGO

```
const [total, setTotal] = useState(0);

return (
  <button onClick={() => setTotal(t => t + 1)}>
    Interessados: {total}
  </button>
);
```

RESULTADO VISUAL

1. Antes do clique: **Interessados: 0**
2. Usuário clica em **Tenho interesse**
3. Depois do clique: **Interessados: 1**

O setter solicita uma atualização. A nova interface usa o estado atualizado.

Imutabilidade: não alterar o original

Com arrays, precisamos produzir um novo valor de estado, em vez de modificar diretamente o array existente.

EVITE

```
eventos.push(novo);
setEventos(eventos);
```



PREFIRA

```
setEventos([...eventos, novo]);
```





A cópia com ... cria um novo array, permitindo que o React reconheça a mudança de referência.

Formulário: um objeto, um atualizador

No componente FormEvento, com useState importado: o name escolhe qual propriedade atualizar.

```
const [form, setForm] = useState({
  titulo: '', descricao: '', data: '',
  local: '', categoria: ''
});
function atualizar(e) {
  setForm({ ...form,
    [e.target.name]: e.target.value });
}
return (
  <>
    <label htmlFor="titulo">Título</label>
    <input id="titulo" name="titulo"
      value={form.titulo} onChange={atualizar} />
    <label htmlFor="local">Local</label>
    <input id="local" name="local"
      value={form.local} onChange={atualizar} />
  </>
);
```

AO DIGITAR NO CAMPO

name = "titulo"

value = "Oficina React"

atualizar(e)

Um novo objeto form.
O título muda; os outros campos são preservados.

id associa a label ao campo. name escolhe a propriedade. value + onChange ligam o campo ao estado.

useEffect: conversar com uma API

O Mural precisa buscar eventos fora do componente. Um efeito permite sincronizar o componente com a API.

CÓDIGO

```
const API_URL = 'http://localhost:3000/eventos';
useEffect(() => {
 const controle = new AbortController();
 fetch(API_URL, { signal: controle.signal })
 .then(r => {
   if (!r.ok) throw new Error('Falha na API');
   return r.json();
 })
 .then(setEventos)
 .catch(e => {
  if (e.name !== 'AbortError')
  setErro('Falha ao carregar os eventos.');
 });
 return () => controle.abort();
}, []);
```

FLUXO DA INFORMAÇÃO

1. A API fornece os eventos.
2. O React guarda os dados recebidos.
3. A lista aparece na interface.



[]: sem dependências reativas. [id]: refaz a busca quando id muda. A limpeza cancela a requisição ao sair da tela ou antes de refazer o efeito.

Efeitos precisam de cuidado

Uma requisição pode demorar, falhar ou precisar ser cancelada.



Carregando

“Carregando eventos...”



Sucesso

A lista de eventos aparece.



Erro

“Não foi possível carregar os eventos.”

`StrictMode`

Em desenvolvimento há um ciclo extra de configuração e limpeza. Isso testa se o efeito pode ser interrompido e refeito. Essa verificação não ocorre em produção.



Uma função de limpeza pode cancelar uma requisição usando AbortController.

Estado derivado: calcular em vez de guardar

A lista filtrada pode ser calculada a partir de eventos, busca e categoria.

EVITE: ESTADO DUPLICADO

```
const [filtrados, setFiltrados] =
 useState([]);

useEffect(() => {
 setFiltrados(eventos.filter(e =>
  e.titulo.toLowerCase()
    .includes(busca.toLowerCase()) &&
   (!categoria ||
     e.categoria === categoria)
  ));
}, [eventos, busca, categoria]);
```

PREFIRA: CALCULAR NA RENDERIZAÇÃO

```
const filtrados =
  eventos.filter(e =>
    e.titulo.toLowerCase()
      .includes(busca.toLowerCase()) &&
    (!categoria ||
      e.categoria === categoria)
 );
```

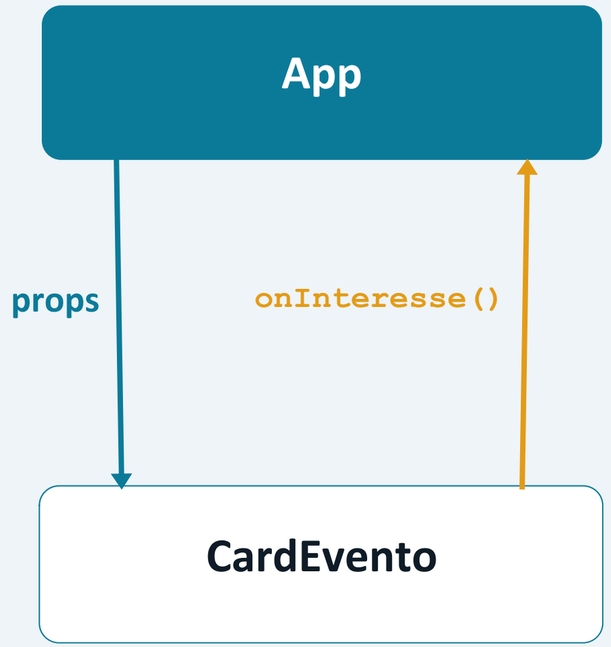


Guardar filtrados causa uma renderização extra e risco de dessincronização. Calculá-los evita duplicar informação. useMemo só ajuda se o cálculo for custoso.

Fluxo de dados: quem controla a mudança?

Os dados descem do componente pai por meio de props. O filho solicita uma ação chamando um callback.

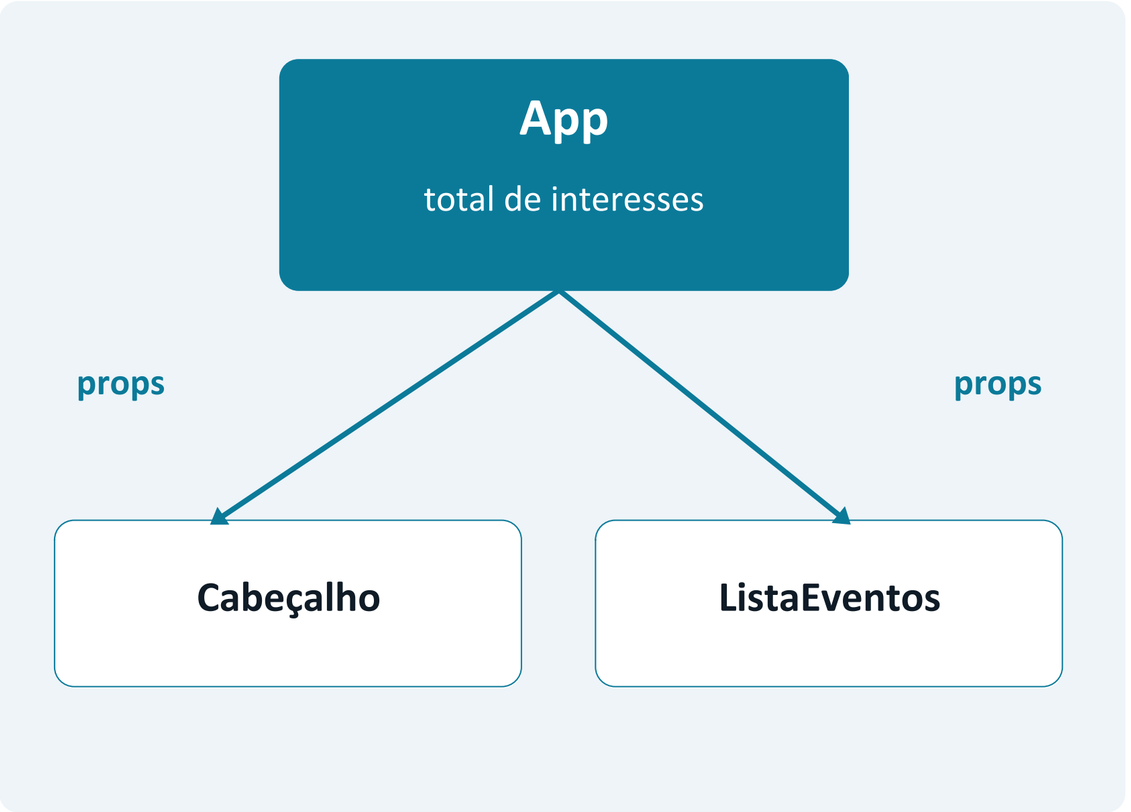
1. **App** guarda o estado.
2. **App** envia dados ao **CardEvento** por props.
3. O usuário clica no botão do card.
4. **CardEvento** chama o callback onInteresse().
5. **App** atualiza o estado.



O pai controla o estado; o filho reage ao clique e solicita a alteração.

Elevar o estado e compartilhar dados

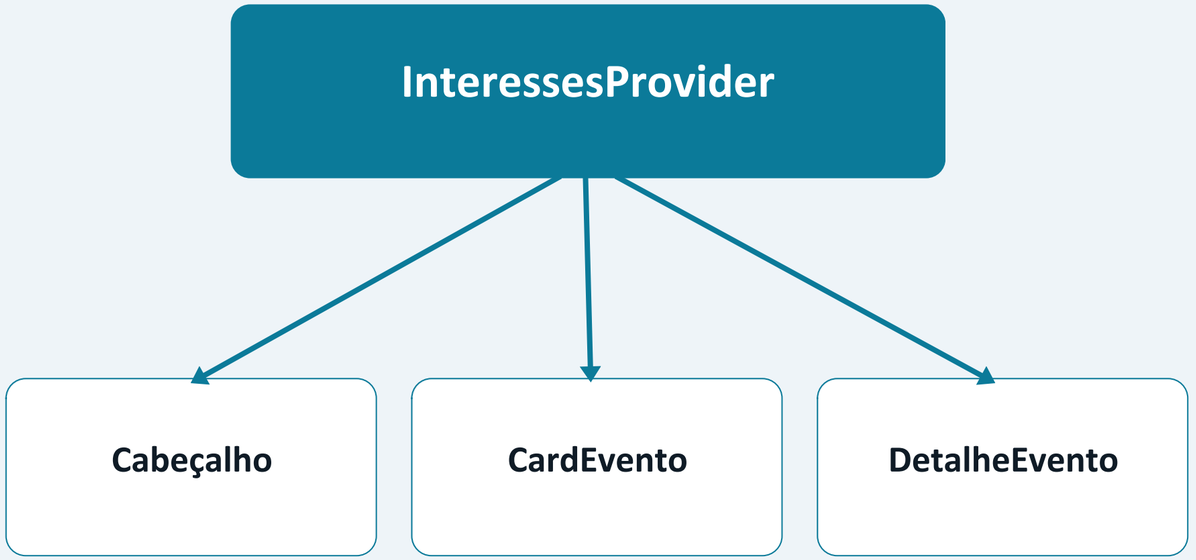
Se o cabeçalho e a lista precisam do mesmo contador, colocamos o estado no ancestral comum, como App.



Repassar props por muitos níveis intermediários é chamado **prop drilling**.

Context API: compartilhar sem repassar

Um Provider disponibiliza informações compartilhadas para diferentes partes do Mural.



Recursos importantes

createContext: cria o contexto.

Provider: disponibiliza os valores.

useContext: permite acessar o contexto.

useInteresses(): hook personalizado que simplifica esse acesso.

Context atende ao Mural, com poucos consumidores e cliques ocasionais. Avalie Redux/Zustand se houver atualizações complexas, frequentes, muitos consumidores ou necessidade de ferramentas de depuração.

Context: Provider e hook em código

InteressesContext.jsx: useState guarda os ids; o Provider disponibiliza o estado e as funções.

```
import { createContext, useContext, useState } from 'react';
const InteressesContext = createContext(null);
export function InteressesProvider({ children }) {
  const [interesses, setInteresses] = useState([]);
  const temInteresse = id => interesses.includes(id);
  function alternar(id) {
    setInteresses(atuais => atuais.includes(id)
      ? atuais.filter(outro => outro !== id)
      : [...atuais, id]);
  }
  return (
    <InteressesContext.Provider
      value={{ interesses, alternar, temInteresse }}>
      {children}
    </InteressesContext.Provider>
  );
}
export function useInteresses() {
  return useContext(InteressesContext);
}
```

1. Estado

interesses é um array de ids.

2. Atualização

alternar devolve um novo array.

3. Acesso

useInteresses lê o valor do Provider.

Consumidores ficam abaixo do Provider. Cada id marcado é compartilhado entre cabeçalho, cards e detalhes.

Context: dois componentes, a mesma fonte

Ambos importam useInteresses do módulo anterior. Marcar ou remover um id atualiza os dois.

CABEÇALHO: CALCULAR O TOTAL

```
function Cabecalho() {
  const { interesses } = useInteresses();
  return (
    <p>Meus interesses: {interesses.length}</p>
  );
}
```

BOTÃO: CONSULTAR E ALTERAR

```
function BotaoInteresse({ evento }) {
  const { alternar, temInteresse } =
    useInteresses();
  const marcado = temInteresse(evento.id);
  return (
    <button type="button"
      onClick={() => alternar(evento.id)}>
      {marcado ? 'Remover interesse'
               : 'Tenho interesse'}
    </button>
  );
}
```

O MESMO PROVIDER ENVOLVE OS CONSUMIDORES

```
<InteressesProvider>
  <Cabecalho />
  <BotaoInteresse evento={{ id: '1' }} />
</InteressesProvider>
```

No Mural, envolver App em main.jsx: o Provider permanece montado ao navegar entre as rotas.

Rotas e ecossistema do Mural

React Router associa a URL a uma tela; a equipe escolhe as ferramentas que complementam a interface.

CAMINHO → COMPONENTE

URL	Tela
/	Lista de eventos
/eventos/novo	Cadastro
/eventos/:id	Detalhe de um evento

RECURSOS DO ROUTER

BrowserRouter: integra o histórico.

Routes e Route: configuram as telas.

Link: navega sem recarregar o documento.

useParams: lê id. useNavigate: redireciona.

```
const { id } = useParams();
```

```
const navigate = useNavigate();
// Após salvar com sucesso:
navigate('/');
```

Ecossistema: Next.js; Redux/Zustand; React Hook Form; Actions do React. Na oficina: Router, Context, campos controlados e fetch.

Formulário: validar, mostrar erro e enviar

Recorte de FormEvento: uma regra do título e o caminho do submit até o POST. Os outros campos seguem as regras de RF3.

VALIDAÇÃO E MENSAGEM POR CAMPO

```
function validar(form) {
  const erros = {};
  if (form.titulo.trim().length < 5)
    erros.titulo = 'Use pelo menos 5 caracteres.';
  return erros;
}
const [tentouEnviar, setTentouEnviar] =
  useState(false);
const erros = tentouEnviar ? validar(form) : {};

<input id="titulo" name="titulo"
  value={form.titulo} onChange={atualizar}
  aria-describedby={
    erros.titulo ? 'erro-titulo' : undefined}
/>
{erros.titulo &&
  <p id="erro-titulo">{erros.titulo}</p>}
```

NO SUBMIT: VALIDAR ANTES DE ENVIAR

```
async function enviar(e) {
  e.preventDefault();
  setTentouEnviar(true);
  if (Object.keys(validar(form)).length) return;
  try {
    const resposta = await fetch(API_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(form)
    });
    if (!resposta.ok)
      throw new Error('Falha ao salvar');
    navigate('/');
  } catch {
    setErroEnvio('Não foi possível salvar.');
  }
}
```

No componente: form/atualizar (slide 19), navigate = useNavigate() e estado erroEnvio. Importar useState, useNavigate e API_URL.

Ligar enviar ao onSubmit do form e mostrar erroEnvio. Na oficina: validar também data, local e categoria; controlar o envio em andamento.

Agora os conceitos vão para a prática

Durante o live coding, veremos como os conceitos trabalham juntos.

PRÓXIMAS ETAPAS

RF1 Listar os eventos obtidos da API.

RF2 Buscar e filtrar os eventos.

Componentes + **Props** + **Estado** + **Efeitos** + **Dados derivados**

OFICINA EM DUPLAS

Checkpoint 1 - Formulário de cadastro (RF3)

Objetivo: Implementar FormEvento com campos controlados, validação e envio à API.

Conceitos: Objeto no estado, onChange por name, validação, POST e useNavigate.

Arquivos: src/componentes/FormEvento.jsx

PASSOS DA DUPLA

1. Criar form com título, descrição, data, local e categoria. Ligar value e onChange a um único atualizar(e), usando name.

2. Criar validar(form): título com 5 ou mais caracteres, data futura, local e categoria obrigatórios. Descrição é opcional.

3. Mostrar cada erro abaixo do campo. Associar label/id e mensagem/aria-describedby.

4. No submit: preventDefault(), validar e bloquear inválidos. Fazer POST JSON, verificar resposta e tratar falhas.

5. Após sucesso, voltar para / com useNavigate. O novo evento deve aparecer na lista.

ESTÁ PRONTO QUANDO

- Vazio, título curto e data de hoje/passada não geram POST.
- Cadastro válido, com descrição vazia, aparece na lista.
- Todos os campos têm label e erros associados.

Dica: preservar os outros campos

```
const atualizar = (e) =>
  setForm({ ...form,
    [e.target.name]: e.target.value })
```

Categorias: Palestra, Workshop, Minicurso e Hackathon. O id é gerado pela API.

Tempo: 18 min

OFICINA EM DUPLAS

Checkpoint 2 - Página de detalhe (RF4)

Objetivo: Abrir um evento pela URL e apresentar os dados obtidos da API.

Conceitos: Rota dinâmica, useParams, useEffect com [id], fetch e Link.

Arquivos: src/App.jsx e src/componentes/DetalheEvento.jsx

PASSOS DA DUPLA

1. Registrar /eventos/:id em App, apontando para DetalheEvento. O título do card já tem o Link.

2. Ler id com useParams. No useEffect com [id], buscar GET /eventos/:id e limpar a requisição ao sair ou mudar o id.

3. Exibir carregamento. Separar sucesso, resposta 404 e outras falhas de requisição.

4. Mostrar título, descrição quando houver, data formatada, local, categoria e identificador.

5. Para 404, exibir “Evento não encontrado”. Incluir Link de retorno à lista nos estados de sucesso e falha.

404: evento ausente. API desligada: falha de conexão. São situações diferentes.

Tempo: 12 min

ESTÁ PRONTO QUANDO

- ☐ Dois títulos diferentes abrem os dados dos ids corretos.
- ☐ Id ausente mostra “Evento não encontrado” e retorno.
- ☐ Voltar usa Link. Carregamento e falha têm mensagem.

Dica: o efeito acompanha o parâmetro

```
const { id } = useParams()
useEffect(() => {
  // buscar o evento de id
}, [id])
```

OFICINA EM DUPLAS

Checkpoint 3 - Interesses com Context API (RF5)

Objetivo: Compartilhar os interesses entre cabeçalho, cards e detalhes.

Conceitos: createContext, Provider, useContext, array imutável e estado derivado.

Arquivos: InteressesContext.jsx, main.jsx, BotaoInteresse.jsx e Cabecalho.jsx

PASSOS DA DUPLA

1. Criar InteressesContext e InteressesProvider com useState([]) para os ids marcados.

2. Implementar temInteresse(id) e alternar(id): adicionar ou remover criando um novo array.

3. Expor o valor pelo Provider e criar useInteresses. Envolver App com o Provider em main.jsx, acima das rotas.

4. Usar o mesmo BotaoInteresse no card e no detalhe. O clique alterna o id e o texto do botão.

5. No cabeçalho, mostrar “Meus interesses: N” com interesses.length. Conferir o estado ao navegar por Link.

Botão: “Tenho interesse” / “Remover interesse”. Persistência após F5 não é exigida.

Tempo: 15 min

ESTÁ PRONTO QUANDO

- ☐ Marcar 2 eventos mostra total 2; remover 1 mostra 1.
- ☐ Card e detalhe exibem a mesma marcação.
- ☐ Interesses continuam ao trocar de rota sem recarregar.

Dica: alternar sem modificar o array

```
setInteresses(atuais =>
  atuais.includes(id)
    ? atuais.filter(item => item !== id)
    : [...atuais, id])
```

Referências bibliográficas

Fontes oficiais consultadas na preparação. Os exemplos foram adaptados ao Mural de Eventos.

HUNT, Pete. Why did we build React? React Blog, 5 jun. 2013. Disponível em: https://legacy.reactjs.org/blog/2013/06/05/why-react.html

LOU, Cheng. One year of open-source React. React Blog, 29 maio 2014. Disponível em: https://legacy.reactjs.org/blog/2014/05/29/one-year-of-open-source-react.html

REACT. Creating a React app. React Documentation, [s. d.]. Disponível em: https://react.dev/learn/creating-a-react-app

REACT. Passing props to a component. React Documentation, [s. d.]. Disponível em: https://react.dev/learn/passing-props-to-a-component

REACT. Quick start. React Documentation, [s. d.]. Disponível em: https://react.dev/learn

REACT. StrictMode. React API Reference, [s. d.]. Disponível em: https://react.dev/reference/react/StrictMode

Referências bibliográficas

Fontes oficiais consultadas na preparação. Os exemplos foram adaptados ao Mural de Eventos.

REACT. useContext. React API Reference, [s. d.].
Disponível em: https://react.dev/reference/react/useContext

REACT. useEffect. React API Reference, [s. d.].
Disponível em: https://react.dev/reference/react/useEffect

REACT. useState. React API Reference, [s. d.].
Disponível em: https://react.dev/reference/react/useState

REACT. You might not need an Effect. React Documentation, [s. d.].
Disponível em: https://react.dev/learn/you-might-not-need-an-effect

REACT. form. React API Reference, [s. d.].
Disponível em: https://react.dev/reference/react-dom/components/form

REACT. input. React API Reference, [s. d.]. Disponível em:
https://react.dev/reference/react-dom/components/input